

Урок 25. Корутины. Практика. Room

20 ноября 2020

В этом уроке рассматриваем интеграцию корутин в Room

Гугл интегрировал поддержку корутин в библиотеки Jetpack. Одной из таких библиотек является Room. Давайте посмотрим, как это работает.

Сначала вспомним, какие у нас есть варианты использования Room без корутин (и RxJava). Предполагается, что вы уже знаете, что такое Room и как его использовать.

В качестве примера у нас есть Dao, возвращающий список комментариев.

```
1 @Dao
2 interface CommentDao {
3
4     @Query("SELECT * FROM comment")
5     fun getAll(): List<Comment>
6
7 }
```

И у нас есть ViewModel с методом fetchData, который вызывается по нажатию кнопки Refresh или какому-либо другому событию. В нем мы планируем получать данные из БД:

```
1 class MainViewModel : ViewModel() {
2
3     // ...
4
5     fun fetchData() {
6         // use room
7     }
8
9     // ...
10 }
```

Чтобы получить данные, надо вызвать этот метод:

```
1 fun fetchData() {
2     val commentsData = db.commentDao().getAll()
3
4     // ...
5 }
```

Но вызывать такой код в main потоке нельзя. Получим ошибку:

error java.lang.IllegalStateException: Cannot access database on the main thread since it may potentially lock the UI for a long period of time.

Надо выносить в фоновый поток. Давайте посмотрим, как можно сделать это с помощью корутин. Так мы лучше поймем, в чем заключается интеграция корутин в Room.

Самое простое решение - обернуть вызов в withContext. Функция withContext позволяет переключить поток выполнения кода, дождаться результата и вернуться обратно в свой поток.

```
1 val commentsData = withContext(Dispatchers.IO) {
2     db.commentDao().getAll()
3 }
```

Помещаем код получения данных в withContext и указываем, что он должен выполняться в IO потоке. withContext выполнит код и вернет нам результат в commentsData.

Но т.к. withContext - suspend функция, мы не можем вызвать ее где угодно.

```
1 fun fetchData() {
2     val commentsData = withContext(Dispatchers.IO) {
3         db.commentDao().getAll()
4     }
5
6     // ...
7 }
```

Этот код будет ругаться, что withContext должен быть вызван в корутине или в suspend функции.

Ок, добавим корутину:

```
1 fun fetchData() {
2     viewModelScope.launch {
3         val commentsData = withContext(Dispatchers.IO) {
4             db.commentDao().getAll()
5         }
6
7         // ...
8     }
9 }
```

Используем viewModelScope, чтобы вызвать корутину. (Подробно о разных видах получения scope я рассказывал в Уроке 23).

Код внутри корутины выполняется в main потоке. Получение данных в withContext будет выполнено в IO потоке. Main поток в это время не будет заблокирован. Как только Room вернет данные, они будут помещены в commentsData, код снова продолжит работу в main потоке и мы сможем сделать все что нужно с полученными данными.

Таким образом мы получаем данные, используя корутины. При этом мы можем еще улучшить читаемость кода. Всю конструкцию withContext с вызовом Room давайте вынесем из корутины в отдельную suspend функцию, которую назовем getComments.

Код теперь выглядит так:

```
1 fun fetchData() {
2     viewModelScope.launch {
3         val commentsData = getComments()
4
5         // ...
6     }
7 }
8
9 suspend fun getComments(): List<Comment> {
10     return withContext(Dispatchers.IO) {
11         db.commentDao().getAll()
12     }
13 }
```

Корутина стала лаконичнее и читабельнее.

Чтобы нам каждый раз не оборачивать получение данных в отдельную suspend функцию с withContext, Room научился делать это сам. В этом и заключается интеграция корутин в Room.

Для этого надо в Dao интерфейсе добавить слово suspend к методу:

```
1 @Dao
2 interface CommentDao {
3
4     @Query("SELECT * FROM comment")
5     suspend fun getAll(): List<Comment>
6
7 }
```

Теперь вызов этой функции в корутине выглядит так:

```
1 viewModelScope.launch {
2     val commentsData = db.commentDao().getAll()
3
4     // ...
5 }
```

И нам не надо создавать никаких отдельных suspend методов с withContext. Room делает это под капотом. Он сам перенесет операцию в фоновый поток и вернет нам данные в поток нашей корутины.

Давайте немного расширим пример. Будем получаемые данные помещать, например, в LiveData:

```
1 val comments = MutableLiveData<List<Comment>>()
2
3 fun fetchData() {
4     viewModelScope.launch {
5         val commentsData = db.commentDao().getAll()
6         comments.value = commentsData
7     }
8 }
```

Метод fetchData запускает корутину, которая получает данные с помощью Room и помещает их в LiveData. Ну а на LiveData уже подписан какой-либо экраный элемент, который отобразит полученные данные.

Flow

Кроме предоставления suspend функций, Room приготовил для нас еще немного интеграции с корутинами. Он умеет возвращать Flow. И это очень важное и полезное умение. Потому что этот Flow будет следить за данными в БД и отправлять их нам, каждый раз когда данные будут изменены. Т.е. это поведение аналогичное Room + Flowable (из RxJava).

Обернем тип возвращаемых данных в Flow:

```
1 @Query("SELECT * FROM comment")
2 fun getAll(): Flow<List<Comment>>
```

Заметьте, что слово suspend здесь уже не нужно. В уроке про Flow я подробно рассказывал, что создание Flow происходит мгновенно и не требует отдельного потока. А вот реальная работа начинается только когда вызвать метод collect.

Вызываем метод getAll, чтобы получить flow.

```
1 val commentsFlow = db.commentDao().getAll()
```

Т.к. getAll - не suspend функция, нет необходимости вызывать ее в корутине. Она может быть вызвана где угодно. Повторюсь, этот код НЕ делает запрос в БД. Он только дает нам Flow объект, который умеет делать этот запрос, когда мы его попросим об этом.

Пример

Давайте посмотрим, как можно использовать Flow в ViewModel.

У нас есть Flow, который подкидывает нам данные по мере их обновления в БД. И мы хотим эти данные складывать в LiveData, из которого они будут на экран попадать. И пользователю даже ничего нажимать не придется. Все будет обновляться автоматически.

Создаем Flow и LiveData:

```
1 val commentsFlow = db.commentDao().getAll()
2 val comments = MutableLiveData<List<Comment>>()
```

Код, который их свяжет, выглядит так:

```
1 viewModelScope.launch {
2     commentsFlow.collect {
3         comments.value = it
4     }
5 }
```

В корутине вызываем collect, чтобы Flow начал свою работу. Его работа заключается в мониторинге БД. Как только данные там меняются, он их нам отправляет. А мы просто передаем их в LiveData.

Эту корутину нам надо запустить один раз при создании ViewModel. Можно сделать это в секции init в классе ViewModel.

```
1 val commentsFlow = db.commentDao().getAll()
2 val comments = MutableLiveData<List<Comment>>()
3
4 init {
5     viewModelScope.launch {
6         commentsFlow.collect {
7             comments.value = it
8         }
9     }
10 }
```

Flow будет работать и держать связь с БД все время пока жива ViewModel и ее viewModelScope.

Выглядит удобно. Но можно сделать еще лучше.

liveData

В Уроке 23 я рассказывал про билдер liveData. Он хорошо подходит под наш сценарий:

```
1 val commentsFlow = db.commentDao().getAll()
2 val comments = LiveData {
3     commentsFlow.collect {
4         emit(it)
5     }
6 }
```

Билдер liveData создает корутину и возвращает LiveData в переменную comments. Он взял на себя код из секции init.

Кроме того, код внутри liveData начнет работу только тогда, когда у LiveData comments появится подписчик. А в прошлом примере он начинал работу в любом случае.

Кода стало чуть меньше. Но и это не предел.

asLiveData

Разработчики гугл понимали, что передача данных из Flow в LiveData будет достаточно часто используемым сценарием, и упаковали его в удобное расширение asLiveData.

В итоге этот же код можно переписать так:

```
1 val comments = db.commentDao().getAll().asLiveData()
```

Всего одна строка кода, и получившаяся LiveData содержит актуальные данные из БД.

Изменение данных

Использовать suspend можно не только для чтения данных, но и их изменения.

Пример:

```
1 @Insert
2 suspend fun insert(comment: Comment)
```

Этот метод теперь можно вызывать в main корутине. Давайте используем его в примере с чтением данных.

Пусть у нас на экране есть список комментариев и возможность добавить новый с помощью кнопки Send. Простая версия кода в ViewModel будет выглядеть так:

```
1  val comments = db.commentDao().getAll().asLiveData()
2
3  fun onSendClick() {
4      viewModelScope.launch {
5          db.commentDao().insert(Comment(...))
6      }
7  }
```

По нажатию кнопки Send вызывается метод onSendClick в котором мы в корутине вызываем метод insert, чтобы добавить новый комментарий в БД.

comments - это LiveData с данными из БД. Как только мы изменим данные методом insert, Flow получит обновленный список и поместит их в comments. А оттуда они уже пойдут на экран. Нам ничего не надо вручную запрашивать.

Это очень удобно. Принцип Single Source of Truth в действии.